Universidade de Aveiro
Departamento de Eletrónica, Telecomunicações e Informática



Information and Coding
2026/27

Lab work n.º 1

Membro	N.º	Participação
Diogo Nascimento	120031	33 3%
Ana Sofia Almeida	113892	33 3%
Beatriz Saraiva	108944	33 3%

October 26, 2026

Contents

1 Introduction **2**

2 Part I **2**
2.1 Reading/Writing a WAV . . . 2
2.2 Plotting Histograms . . . 3
2.3 Audio Quantization . . . 3
2.4 Comparing Audios . . . 3
2.5 Audio Effects . . . 3

3 Part II - Lossless Audio Codec **4**
3.1 State-of-the-Art Evaluation . . . 4
3.2 Our Implementation . . . 5
3.3 Model Comparison . . . 6
3.4 Final Adjustments . . . 7

4 Part III - Lossy Audio Codec **8**
4.1 State-of-the-Art Evaluation . . . 8
4.2 Our Implementation . . . 8
4.3 Model Comparison . . . 9
4.4 Final Adjustments . . . 10

5 Manual - How to Run **11**
5.1 Building the Project . . . 11
5.2 Using the Codecs . . . 11
5.2.1 Lossless Audio Codec . . . 11
5.2.2 Lossy Audio Codec . . . 12

Introduction

This project comprises of 3 distinct modules. Each module takes audio files, in WAV format, and extracts, modifies and displays their data. Alongside the explanations in this report, additional information can be found in our project repository at `https://github.com/DCANascimento/IC_Trabalho_1`.

Part I

For the first part, we started by learning the intricacies of audio waveform manipulation by creating some simple files. These include:

- `wav_hist:` Given an audio WAV file, this program displays an histograms for various of its channels.
- `wav_quant:` This program takes an audio file and creates a new version that uses less bits (through quantization).
- `wav_cmp:` This program takes a quantized audio and its original form, and compares them using metrics like the average mean squared error, the maximum absolute error and the signal-to-noise ratio.
- `wav_effects:` After choosing an audio file, this last program gives a selection of audio effect options, applies them to the given audio and plays them.

We began the project by testing operations on the audio WAV files by using , since we were more used to it, and it helped us get a better understanding of the audio format. Afterwards, we decided to take our implementations and convert them to C, since our priority was processing times, we wanted a language that gave us more direct control over memory allocation. However, by the end, we translated the code done to C++, since it was simpler to work with, gave us similar performance results, and simplified the whole project, since for the codecs later on C++ was the language we already started with.

The main function files are in the /src folder, while the execution files that are ran in the terminal exist in the /programs folder.

2.1 Reading/Writing a WAV

In the first step, we immediately encountered a major difference between MATLAB and C. In MATLAB there are functions that automatically extract the data from the WAV in an array with 2 rows, each one already representing a channel, while in C, all that we get is a byte array, so we have to extract the relevant chunks:

1. **RIFF:** Identifying by the RIFF header lets us extract the size of the file. The header is a 12 byte array where the first 4 bytes represent its RIFF ID, the next four are the file size, and the last 4 are a WAVE ID. This wouldn't be used now, but would be useful for the later codecs.
2. **FMT:** The FMT chunk is the first important part, since it gives us important metadata such as the number of channels, the sample rate, the bits per sample (to ensure its 16-bit), the audio format (which we assumed by default would be PCM), the byte rate and block align.
3. **DATA:** The data chunk tells us the total size of the data. With it we can calculate the total number of samples, and allocate memory accordingly. It is also in it that we have the actual PCM audio samples that we will work with.

2.2 Plotting Histograms

This part of the project simply served to help us understand the structure of the WAV files we would be working with as a baseline. We simply read the WAV files, take their samples, separate the audio tracks into left and right, calculate the MID and SIDE channels (by taking the average and difference of the two, respectively), and create the histograms, which display the distribution of sample amplitudes for each channel.

2.3 Audio Quantization

For the quantization of a given audio file, we choose a value between 1 and 16 bits (16 bits is the original format, while 1 is the bare minimum). Then taking the number of bits selected and the total number of samples, we can calculate the total expected size of the new quantized audio array and, one by one, calculate the quantized value of each sample by rounding the value of said sample down.

However, for this part of the project, since we end up writing the WAV files using a 16 bit representation, independent of any quantization done, the actual size of the new file is actually the same.

For the WAV writing part, we simply do the reverse of the reader. We create the RIFF header (RIFF ID + file size + WAVE ID), we recreate the FMT chunk and the DATA chunk (which in theory have the same values, since the only modification done was to the data size in the DATA chunk, which was then reverted).

2.4 Comparing Audios

To compare an audio file to its quantized form, we take both audio and start by checking if they share relevant parameters, like number of channels, number of samples and sample rate. After enduring this, the program computes metrics including the mean squared error (MSE), maximum absolute error, and the Signal-to-Noise Ratio (SNR) in decibels by evaluating signal power against error accumulation.

2.5 Audio Effects

This component implements a suite of time-domain digital audio effects to dynamically alter the sound profile. By utilizing delay lines, feedback and feedforward loops, and modulation techniques (such as tremolo, vibrato, chorus, flanger, and Doppler shifts), the system transforms the waveform frames. It then normalizes the final output peaks to prevent signal clipping before writing the processed data back into a standard WAV format.

Part II - Lossless Audio Codec

Moving to the codecs, the ***Lossless Audio Codec*** is a program that takes an audio file and compresses it, to as small of a size as possible, while making all the information in the original recoverable through decompression.

3.1 State-of-the-Art Evaluation

We first decided to make a general evaluation of other lossless audio codecs, in order to better understand other implementation, and potentially figure out what we could learn from. After some research we ended up with the information displayed in Table 3.1.

Table 3.1: Lossless audio codec comparison

Codec / Compressor	Pros	Cons
FLAC	• As it was designed for streaming, it has an independent frame structure that facilitates streaming and random access. • Excellent compatibility. • Effectively supported by any modern readers. • Open source.	• Does not support any hybrid compression mode. • Is limited to PCM audio format • Variable compression efficiency may make it less effective than other competitors depending on the audio
WavPack	• Similar to better compression than FLAC. • Very fast encoding and decoding. • Supports an hybrid mode. • Supports streaming. • Open source.	• Less widely compatible than FLAC
TAK	• Compression is more consistently efficient than FLAC • Very fast decoding. • Generally very efficient.	• Smaller ecosystem. • Lower support outside of Windows. • Not open source.
Zstd	• Extremely fast • Excellent compression ratio for generic data • Open source • Widely used in backups, databases and Linux	• Missing direct playback, streaming, musical metadata and music compatibility
LZMA (7-Zip,xz)	• Very high compression (more so than Zstd) • Great for archival • Open source	• Slower than Zstd • Missing the same features as Zstd

3.2 Our Implementation

This part of the work was split into a few submodules:

- **wav.cpp** allows us to read/write a WAV file. From a WAV file, besides the PCM audio samples that are later encoded, we also take metadata, such as sample rate, number of channels, bits per sample and audio format. During decoding, this data is written back in, alongside the original audio samples.
- **predictor.cpp** takes the raw samples values, and by applying a prediction algorythm based on previous samples, it can reduced large amplitude values to smaller, residual ones, to increase compression.
- **zigzag.cpp** applies the zigzag method to convert signed integers into unsigned integers, and vice versa. This simply serves to enable rice encoding later on, as it only takes in positive integers.
- **rice.cpp** takes the values transformed by the zigzag, and converts them to bits, and vice versa. For this encoding method, a difficulty that arised was the decision of the **k** variable. This variable is what decides the length of the bit representation of each samples. However, there is no single value of **k** that always garantees the shortest possible encoding, and sometimes even the best value ends up longer than the original PCM stream. Given this, our code has a range of **k** values that are tested and compared (including the original samples), and the most efficient compresion variable is kept and used.
- **bitstream.cpp** takes the values transformed by the zigzag method, and converts them to bytes. Besides the compressed sample values, it also converts and stores important data for decoding, like the **k** variable obtained in the rice method.
- **format.cpp** grabs the data prepared by the bistream, and writes out the CLSS file. It also contains the logic for the reading of the CLSS.

Below, in Fig. 3.1, is a diagram we made for clarity. Later into development, the prediction logic was moved to the `block_codec.cpp` file, and the `zigzag.cpp` and `rice.cpp` files were called from `block_codec.cpp`. In short, the compression and decompression code process can be described with the following sequences:

Compression pipeline:

`main.cpp` (`runApplication()` → `compressFile()`)
→ `wav.cpp` (`loadWavInfo()` : samples, metadata, `WavLayout`)
→ `block_codec.cpp` (`encodeBlockPayload()` : block partitioning and encoding selection)

- `makeResidualValues()` : channel modes, first-/second-order prediction, ZigZag
- `chooseRiceK()` → `riceEncode()` → `BitWriter`
- Raw PCM per block, if Rice does not reduce the size

→ `format.cpp` (`writeCompressedFile()`)

Decompression pipeline:

`main.cpp` (`decompressFile()`)
→ `format.cpp` (`readCompressedFile()` : header validation and prefix, suffix, and payload extraction)
→ `block_codec.cpp` (`decodeBlockPayload()` : record parsing and block decoding)

- Rice block: `riceDecode()` using `BitReader`
  → `zigzagDecode()` → inverse prediction and, if needed, mid/side decoding
- Raw PCM block: direct sample reconstruction

→ `wav.cpp` (`saveWavPreservingLayout()`)

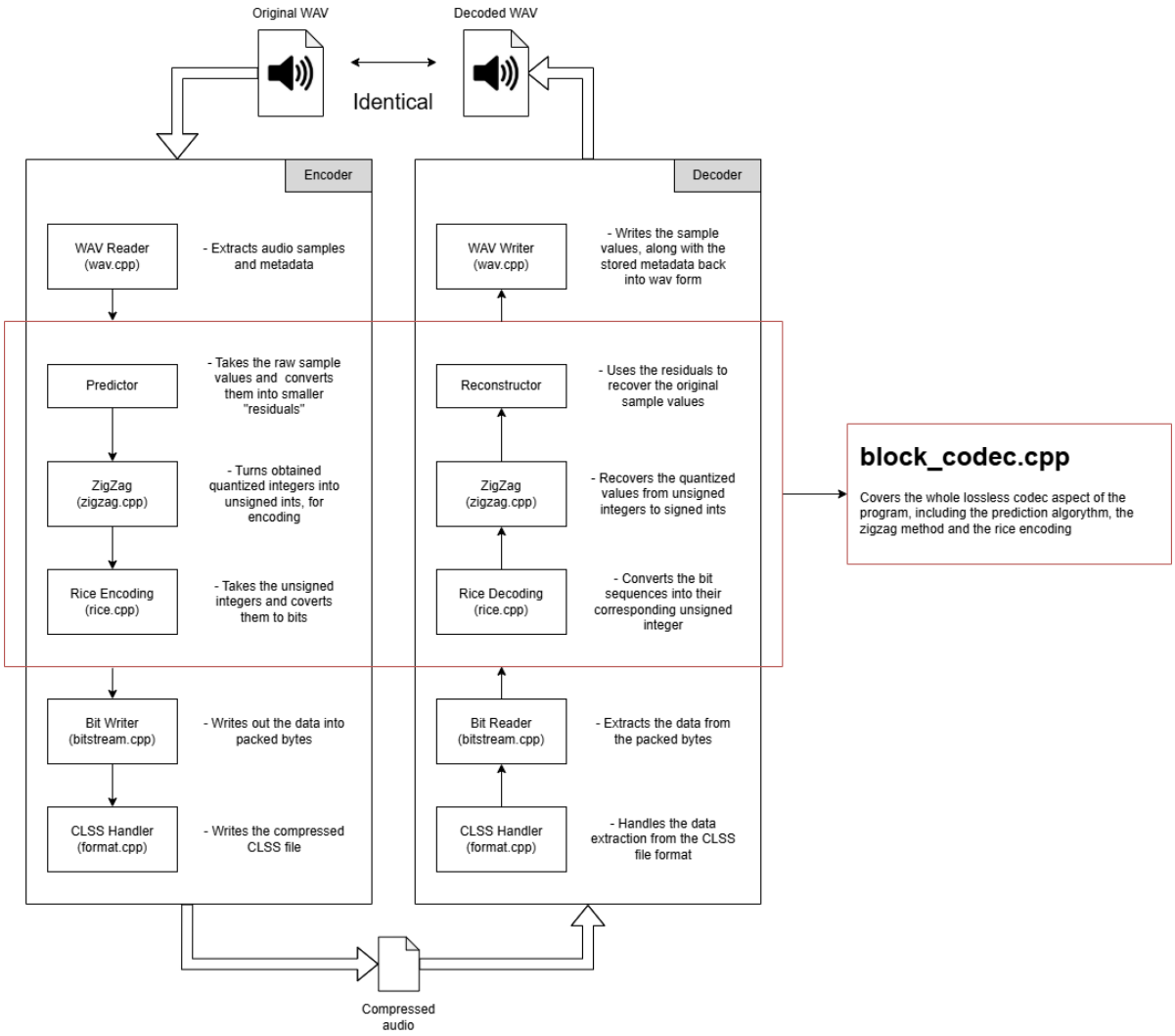


Figure 3.1: Diagram of the Encoding/Decoding Pipeline of the lossless codec

3.3 Model Comparison

To compare our codec with state-of-the-art codecs, all of them were evaluated on the `Audio_Datasets` folder provided by the teachers. It contains 20 WAV files

We have chosen the following configurations for each model:

- **CLSS:** Our current configuration
- **FLAC:** (v1.5.0) -5, single thread
- **WavPack:** (v5.9.0) normal mode, single thread
- **TAK:** (v2.3.3) -p2, single thread
- **Zstandard:** (v1.5.7) -3, single thread
- **LZMA:** (v5.8.4) -6, single thread (using XZ Utils)

In the table 3.2, the memory peak collumns considers the peak during compression and during decompression (left and right, respectively). Our model seems to lag behind in compression efficiency and file size. It uses simpler techniques and experiences higher execution time and memory overhead due to exhaustive parameter searching kept entirely in RAM. However, it seems to outperform generic compressors in overall compressed size. It creates smaller files than Zstandard (which is faster but less efficient) and beats out LZMA in compression ratio while drastically beating LZMA's execution time and memory usage.

Table 3.2: Comparison of lossless audio compression models

Codec	Total output	WAV	Reduction	Comp. (s; MB/s)	Decomp. (s; MB/s)	Memory peak
CLSS	60 424 392 B	48,31%	51,69%	8,884; 14,08	5,112; 24,47	21,38 / 21,15 MiB
FLAC	49 351 027 B	39,46%	60,54%	3,902; 32,05	3,358; 37,24	9,12 / 4,23 MiB
WAVPack	50 609 404 B	40,47%	59,53%	6,384; 19,59	6,455; 19,38	7,02 / 6,12 MiB
TAK	46 832 472 B	37,45%	62,55%	4,812; 25,99	4,656; 26,86	7,44 / 7,38 MiB
Zstandard	82 691 717 B	66,12%	33,88%	2,765; 45,24	2,114; 59,17	29,14 / 9,65 MiB
LZMA	61 419 425 B	49,11%	50,89%	54,226; 2,31	5,818; 21,50	97,24 / 12,27 MiB

3.4 Final Adjustments

Given our obtained results, we decided on some changes that could potentially improve our compression results:

1. **MID/SIDE channels:** By taking the difference between the left and right channels (assuming stereo), as well as the average of the two, we can potentially obtain smaller values to the original ones, increasing compression rate. However, like with Rice encoding, if the resulting values are larger, we can choose to keep the originals.

2. **Predictor adjustment:** By using a second order predictor, as opposed to first order, we can model smooth waveforms by tracking signal trends and slopes rather than assuming a flat line, resulting in significantly smaller residuals and improved compression efficiency

3. **Block encoding:** Before, we would apply the whole enconding strategy to the entire sample field, all with the same parameters. Now, we have decided to split the samples into blocks, and each block now has its own compression parameters. So each block can either have L/R or MID/SIDE transformation, choose between first or second order predictor, define a favorable **k** variable for rice encoding, and decide whether to do said encoding or keep the raw PCM samples (for all of these, each block is tested at runtime and the best result is chosen)

After these changes were applied, we observed a significant improvement in the general statistics of compression, as seen in Table 3.3

Table 3.3: Performance comparison between two versions of the lossless audio codec

Performance Metric	Codec Version 1	Codec Version 2
Compression Time	0.654238 s	0.628579 s
Processing Speed	7.91264 MB/s (PCM)	8.23564 MB/s (PCM)
Original Size	5,176,796 bytes	5,176,796 bytes
Compressed Size	3,813,828 bytes	3,656,438 bytes
Compressed / Original	73.67%	70.63%
Compression Ratio	1.36:1	1.42:1

Note: Version 2 demonstrates both faster execution time and a more efficient compression ratio compared to Version 1.

Part III - Lossy Audio Codec

Lastly, for the third part, we implemented a ***Lossy Audio Codec*** which, unlike lossless, focuses on a higher compression rate by sacrificing a portion of the original information.

4.1 State-of-the-Art Evaluation

Like how it was done for the lossless variant, before developing our lossy audio codec we began with an evaluation of other existing codecs. The results are displayed below, in Table 4.1.

Table 4.1: Lossy audio codec comparison (based on raw mathematical SNR, bitrate, and resource benchmarks)

Codecs	Pros	Cons
MP3 (LAME)	• Universal compatibility across virtually every legacy and modern hardware/software player. • Well-established, mature ecosystem.	• Lower compression efficiency and poorer BD-rate performance compared to modern codecs. • Higher overall program memory footprint when running via wrapper suites like FFmpeg.
AAC (FFmpeg)	• Industry standard for streaming, web, and Apple ecosystems. • Better rate-distortion efficiency than MP3.	• Lower raw mathematical SNR than our custom transform codec in direct unweighted benchmarks. • Higher memory overhead during runtime.
Opus	• State-of-the-art perceptual quality for speech and music, especially at low bitrates. • Highly optimized for real-time communication and streaming.	• Lower raw mathematical SNR in direct benchmarks because it heavily targets psychoacoustics rather than raw wave matching. • Fixed 48 kHz internal decoding pipeline requires resampling overhead.

Note: Raw Signal-to-Noise Ratio (SNR) does not equal perceived quality; standard codecs like MP3, AAC, and Opus utilize advanced psychoacoustic models to remove inaudible frequencies and place noise where it cannot be heard, meaning they may sound better to the human ear despite lower mathematical SNR scores.

4.2 Our Implementation

Similarly to the previous codec, this one was split into:

- **wav.cpp** still handles WAV file reading and writing.
- **mdct.cpp** + **fft.cpp** apply the sine window, handles block overlapping, and computes the MDCT/DCT efficiently by folding the input into a DCT-IV using an internal FFT.
- **transform.cpp** selects the transform and validates the block size.
- **codec.cpp** contains more aggregated operations, such as the quantizer (later implemented with deadzone + centroid offsets), per-block Mid/Side decision and context-adaptive Rice parameter (also implemented in a later version), file format LSY6 and rate control.

- **bitstream.hpp** provides low-level bit manipulation via 'BitWriter' and 'BitReader'. It packs variable-length Rice codes, the 'last' non-zero position markers, and stereo Mid/Side flags tightly into the output byte stream. It also contains calls the functions for the zigzag and rice encoding operations.
- **bytes.hpp** provides lightweight utility functions and helper classes for low-level binary file I/O and little-endian data manipulation in memory.
- **wav_lossy_enc.cpp / wav_lossy_dec.cpp / wav_snr.cpp** are the three main programs for execution.

A model of the first implementation of the pipeline is shown in Figure 4.1

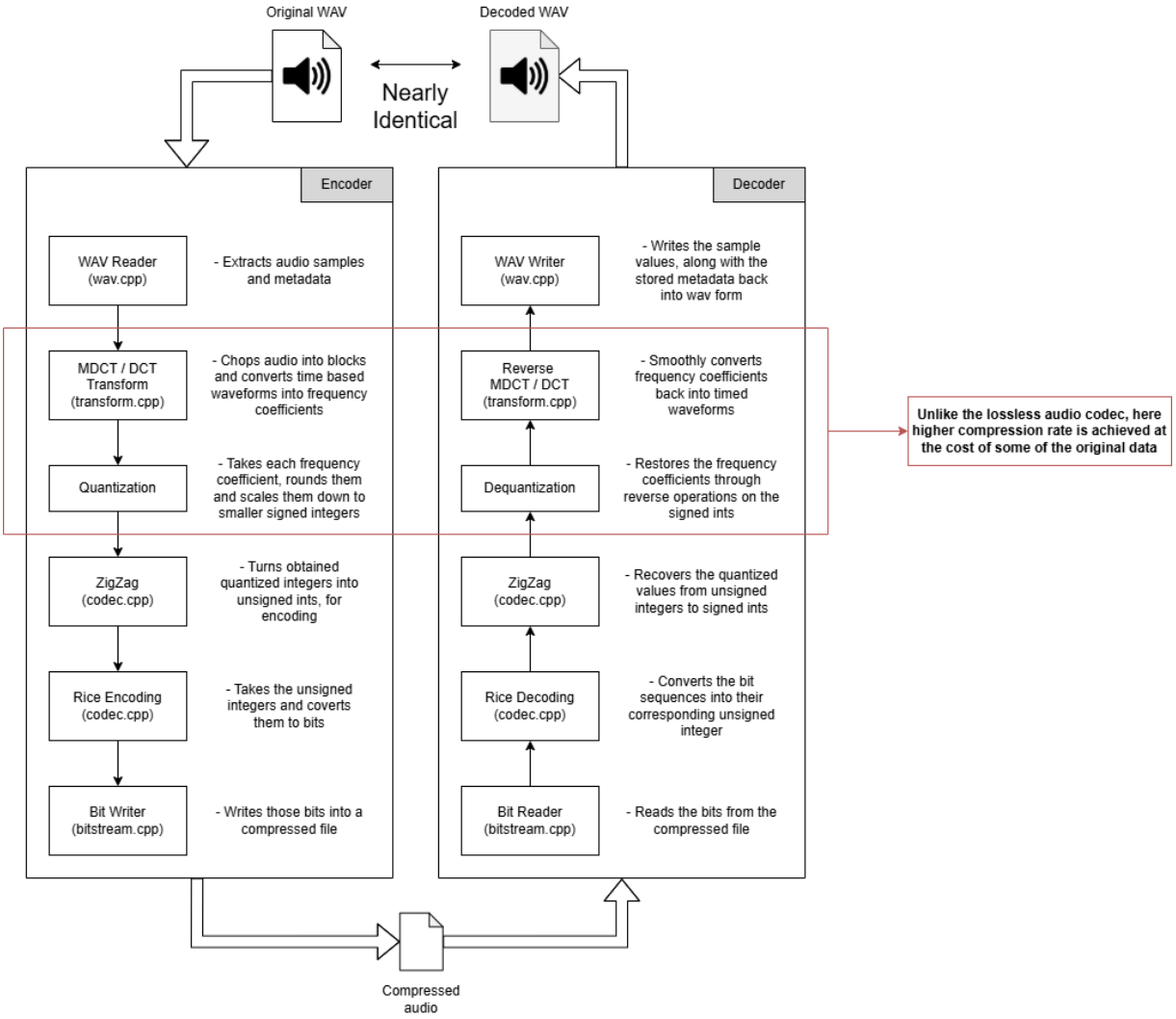


Figure 4.1: Diagram of the Encoding/Decoding Pipeline of the lossy codec

4.3 Model Comparison

Next came the model comparison stage, and in Table 4.2 and Table 4.3, we can observe the results obtained from the 7 audio files provided (in the AudioFiles folder). Our codec ended up performing drastically better in terms of memory usage than its competitors, as well as doing better decoding time. However, and most importantly, it achieves a higher SNR per bitrate, which was what we were hoping for.

Table 4.2: Comprehensive performance, memory, and quality comparison at 128 kbps

Codec / Config	Enc. Time (s)	Dec. Time (s)	Enc. Mem (MB)	Dec. Mem (MB)	SNR (dB) at 128k	BD-Rate
Ours (-b 128)	0.92	0.041	17.0	14.1	33.56	Baseline
MP3 (LAME)	0.35	0.061	51.9	52.1	19.58	+148 %
AAC (FFmpeg)	0.45	0.061	52.1	52.5	18.63	+196 %
Opus	0.24	0.103	52.3	52.0	20.36	+215 %

Table 4.3: Comparison of target vs. actual bitrates and compression ratios across audio codecs

Target	Ours (v2)		MP3 (LAME)		AAC (FFmpeg)		Opus	
(kbps)	kbps	Ratio	kbps	Ratio	kbps	Ratio	kbps	Ratio
32	32,0	44,10	32,2	43,87	33,1	42,64	33,6	42,54
48	48,0	29,40	48,2	29,28	49,6	28,50	50,4	28,33
64	64,0	22,05	64,2	21,97	65,5	21,58	67,3	21,21
96	96,0	14,70	96,3	14,65	97,3	14,53	100,6	14,19
128	128,0	11,03	128,4	10,99	129,8	10,89	134,4	10,62
160	160,0	8,82	160,6	8,79	161.5	8,75	167,5	8,51
192	192,0	7,35	192,7	7,32	194,1	7,28	199,7	7,13
256	256,0	5,51	256,9	5,49	257,2	5,50	264,4	5,38
320	320,0	4,41	321,1	4,39	314,9	4,50	325,8	4,36

4.4 Final Adjustments

Like with the lossless encoder, we were already expecting that block encoding would end up being implemented, to improve our compression results, as well as splitting both channels (in the case of stereo) into MID and SIDE channels. So for the final version of the lossy codec, besides those changes, we have also identified and implemented other possible improvements:

1. **Quantization and centroid reconstruction:** We replaced simple uniform scaling with an advanced deadzone quantizer, which effectively eliminates faint background noise, and paired it with centroid reconstruction parameters, which reduce the error that arises from the deadzone quantization.
2. **Context based Rice encoding:** While before we used a single global parameter for an entire block, often resulting in inefficient bit distribution, we now dynamically calculate and predict an optimal k parameter for every single coefficient.

After implementing those changes, we observed a noticeable improvement, as seen in Table 4.4, specially from the context-adaptive Rice parameter k. Each row of the table is cumulative, and after all the changes done we obtained a BD-rate improvement of about -32.7% and BD-SNR improvement of +8.056dB.

Table 4.4: Incremental performance improvements (BD-rate and BD-SNR)

Improvement (Cumulative)	BD-rate (bits at equal SNR, %)	BD-SNR (SNR at equal bitrate, dB)
v1 + context-adaptive Rice parameter	-30.44	+7.396
+ adaptive Mid/Side	-1.98	+0.493
+ deadzone and centroid	-1.27	+0.248

Manual - How to Run

This project was developed for Ubuntu/Debian-based systems. Before running any scripts, ensure that the following dependencies are installed:

- `gcc` / `g++` (C11 and C++17 support)
- `make` and `pkg-config`
- GTK4 (`libgtk-4-dev`), required only for the graphical interface
- At least one audio player: `paplay` or `aplay` for GUI and terminal playback; `ffplay` is an additional fallback for the GUI
- MATLAB, required only for the `.m` prototypes
- `cmake` >= 3.20, only for the lossy codec (Part III)
- `python3`, only for the lossy codec's test and measurement scripts

Most of the required dependencies can be installed using the following command:

```
sudo apt update
sudo apt install build-essential make pkg-config libgtk-4-dev \
    pulseaudio-utils alsa-utils ffmpeg
```

5.1 Building the Project

To build the exercises in Part 1, open a terminal in the project's root directory and run the following commands:

```
cd Project
make
```

After the build completes successfully, the executables should be available in the `Project` director, to run the executables as follows:

```
// WAV Histograms
./wav_hist input.wav

// WAV Quantization
./wav_quant input.wav output.wav --bits n

// WAV Comparison
./wav_cmp input.wav input_quantized.wav

// WAV Audio Effects
./wav_effects
```

5.2 Using the Codecs

5.2.1 Lossless Audio Codec

For the lossless codec, first extract the zip file "LosslessCodec_FINAL.zip". Inside of it, use the following commands to build and run the program:

```
cmake -S . -B build-user
cmake --build build-user --config Release
```

This creates the encoder and decoder, which can be tested using:

```
// Encoder
./build-user/wav_lossless_enc input.wav compressed.clss

// Decoder
./build-user/wav_lossless_dec compressed.clss restored.wav
```

Note: For the final version of the codec, the zip file may need to be extracted using `git lfs`. For that, run these commands:

```
git lfs install
git lfs pull
```

5.2.2 Lossy Audio Codec

To run the lossy codec, simply go to the "lossy" folder, build the lossy codec using the cmake file and test the program. From the root, the commands are as follows:

```
cd lossy
cmake -S . -B build
cmake --build build
```

This will create the encoder, decoder, and an auxiliary file to compare the original file with the compressed version. To then run the program use:

```
// Encoder
./build/wav_lossy_enc [options] input.wav compressed_file

// Decoder
./build/wav_lossy_dec compressed_file output.wav

// Comparator
./build/wav_snr original.wav decoded.wav
```

For the encoder, the options that can be run include:

- **-q <step>** | to set a quantization step (>= 0.01);
- **-b <kbps>** | to set a target bitrate;
- **-n <size>** | to set a block size;
- **-t <type>** | to choose between "mdct" or "dct" transform;
- **-z <r>** | to set the value for quantizer rounding;
- **--no-centroid** | to decode at the middle of each interval;
- **--no-ms** | for stereo, to code using L/R, instead of MID/SIDE
- **-v** | to print statistics in the end.